VEPHLA UNIVERSITY

DEPARTMENT OF CYBERSECURITY

GRADUATION PROJECT: SECTION 3: SYSTEM OPERATIONS, OFFENSIVE AWARENESS, AND SECURITY GOVERNANCE

Target System: `testphp.vulnweb.com`

SUBMITTED BY: OLANIYI IBRAHIM ABIODUN

STUDENT ID: VEPH/20B/CY070

DATE: January 9th, 2026

SUPERVISOR: Dev Richard

Table of Contents

Executive Summary **3**
Question 3A: System Interaction Using Kali Linux **4**
Step-by-Step Installation of a Virtual Machine 4
Step-by-Step Installation of Kali Linux 5
How Kali Linux is Integrated 6
Practical System Interaction (CLI Commands) 7
Question 3B: Website Hardening Recommendations **10**
Core Infrastructure Hardening 10
Security Header Implementation Plan 11
Question 3C: Security Policy Design **12**
Password Policy 12
Acceptable Use Policy (AUP) 12
Incident Reporting Policy 12
Policy Alignment Table 13
Question 3D: Quantitative Risk Analysis **14**
Asset Selection: Customer Database 14
Risk Calculations 14
Influence on Security Investment Decisions 15

Executive Summary

This final phase of the assessment shifts from identifying vulnerabilities to establishing a **sustainable security posture** through system hardening, policy design, and quantitative risk management.

I have approached this phase by first documenting the foundational integrity of the testing environment. By demonstrating proficient interaction with the **Kali Linux** ecosystem, I have established that the technical tools used for this audit are managed within a controlled, professional framework. This ensures that all findings are reproducible and that remediation steps are tested in an isolated, secure "sandbox" before deployment to production.

- **Infrastructure Hardening:** I have moved beyond the discovery of flaws to propose specific, actionable technical controls. This includes transitioning to **HTTPS**, updating core backend components, and implementing modern security headers to neutralize the attack vectors identified in Section 2.
- **Governance and Policy Design:** Recognizing that technical fixes are temporary without administrative oversight, I have drafted core security policies tailored to the organization's needs. These policies—covering passwords, acceptable use, and incident response—align with the **CIA Triad** to ensure long-term operational resilience.
- **Quantitative Risk Validation:** To bridge the gap between technical risk and business impact, I have performed a **Quantitative Risk Analysis**. By calculating the **Annual Loss Expectancy (ALE)**, I provide a clear financial justification for security investments, moving the conversation from "technical bugs" to "business risk management."

This section serves as the final bridge in the Cyber Kill Chain, moving from the perspective of an attacker performing reconnaissance to a defender building a hardened, resilient infrastructure.

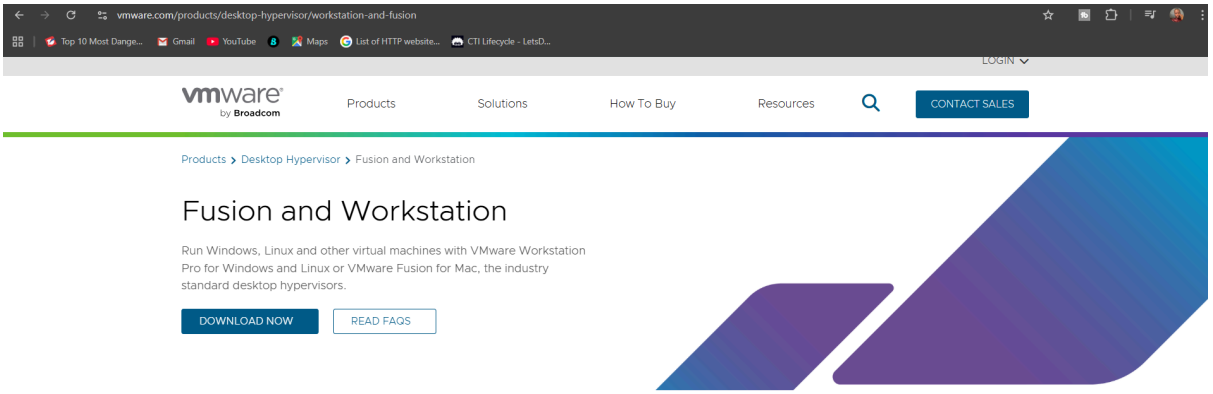
Question 3A: System Interaction Using Kali Linux

For this assessment, I utilized **VMware Workstation** to host my security environment. Virtualization allows me to run a guest operating system (Kali Linux) in a secure, isolated container while utilizing the physical resources of my host machine.

Step-by-Step Installation of a Virtual Machine

Step 1: Download the Installer

1. **Visit the Official Site:** Go to the [VMware Customer Connect]() download page.
2. **Select the Version:** Look for **VMware Workstation Player for Windows 64-bit Operating Systems**.
3. **Download:** Click **Download Now** and save the `.exe` file to your Downloads folder.



Step 2: Run the Installation Wizard

1. **Launch:** Double-click the downloaded `.exe` file. If prompted by **User Account Control**, click **Yes**.
2. **Welcome Screen:** Click **Next** to begin.
3. **License Agreement:** Read and **Accept** the terms, then click **Next**.
4. **Custom Setup:** * Ensure the **Enhanced Keyboard Driver** is checked (this helps with better keyboard shortcut support in Kali).
    - Choose your installation path (default is usually fine) and click **Next**.

5. **User Experience:** You can uncheck "Check for product updates" and "Join VMware Customer Experience" if you prefer privacy, then click **Next**.
6. **Shortcuts:** Keep "Desktop" and "Start Menu" checked for easy access, then click **Next**.

Step 3: Finalize and Launch

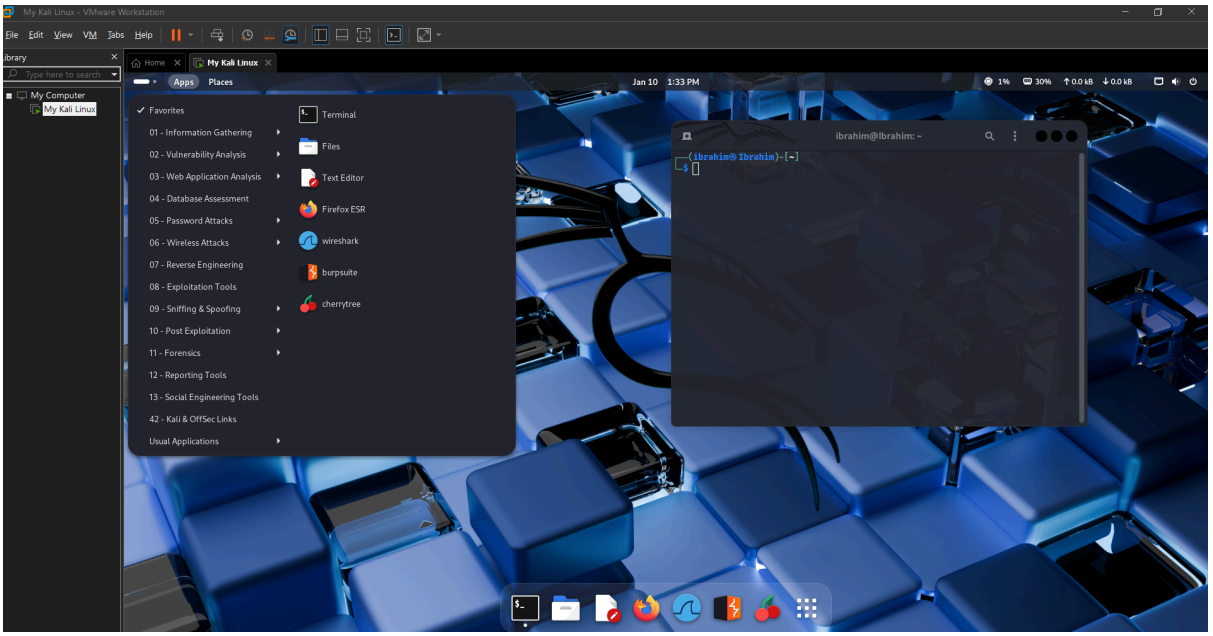
1. **Install:** Click the **Install** button. This may take a few minutes as it configures virtual network adapters.
2. **Finish:** Once complete, click **Finish**. You may be prompted to **Restart** your computer; do this to ensure the virtual drivers are properly loaded.
3. **First Launch:** Open VMware from your desktop. Select **"Use VMware Workstation Player for free for non-commercial use"** and click **Continue**.

Step-by-Step Installation of Kali Linux

As a professional analyst, I followed these steps to ensure a stable and performant testing environment:

1. **Preparation and Resource Check:** I downloaded the latest **Kali Linux ISO** from the official OffSec repository. Before starting, I verified that my host system had sufficient hardware overhead to support the VM without performance degradation.
2. **VM Creation Wizard:** Within VMware Workstation, I selected **"Create a New Virtual Machine"** and chose the **"Typical"** configuration. I pointed the installer to the downloaded Kali ISO file.
3. **Hardware Customization:** This is a critical step for security tools like Nessus. I manually adjusted the virtual hardware settings to include:
    - **Memory:** Allocated **4GB of RAM** (minimum) to ensure the Nessus engine can compile plugins efficiently.
    - **Processors:** Allocated **2 CPU cores** to handle multi-threaded scanning tasks.
    - **Storage:** Allocated **60GB of Disk Space**, choosing "Store virtual disk as a single file" for better performance.

4. **Network Configuration:** I set the network adapter to **NAT (Network Address Translation)**. This allows the VM to share the host's IP address for internet access (required for plugin updates) while keeping the VM on a private internal subnet.
5. **Finalizing OS Setup:** I powered on the VM and followed the Kali graphical installer to set the language, timezone, and **non-root user credentials**.



How Kali Linux is Integrated

Kali Linux is integrated into this project as the **centralized security workstation**. Unlike a standard OS, Kali is pre-configured with the environment variables and dependencies required for the tools used in Sections 1 and 2 (Nmap, Burp Suite, and Wireshark).

The integration is finalized by installing **VMware Tools**. This allows for professional-grade interaction, such as:

- **Clipboard Sharing:** Allowing me to copy technical findings directly into my report.
- **Shared Folders:** Facilitating the secure transfer of logs and packet captures between the guest and host.

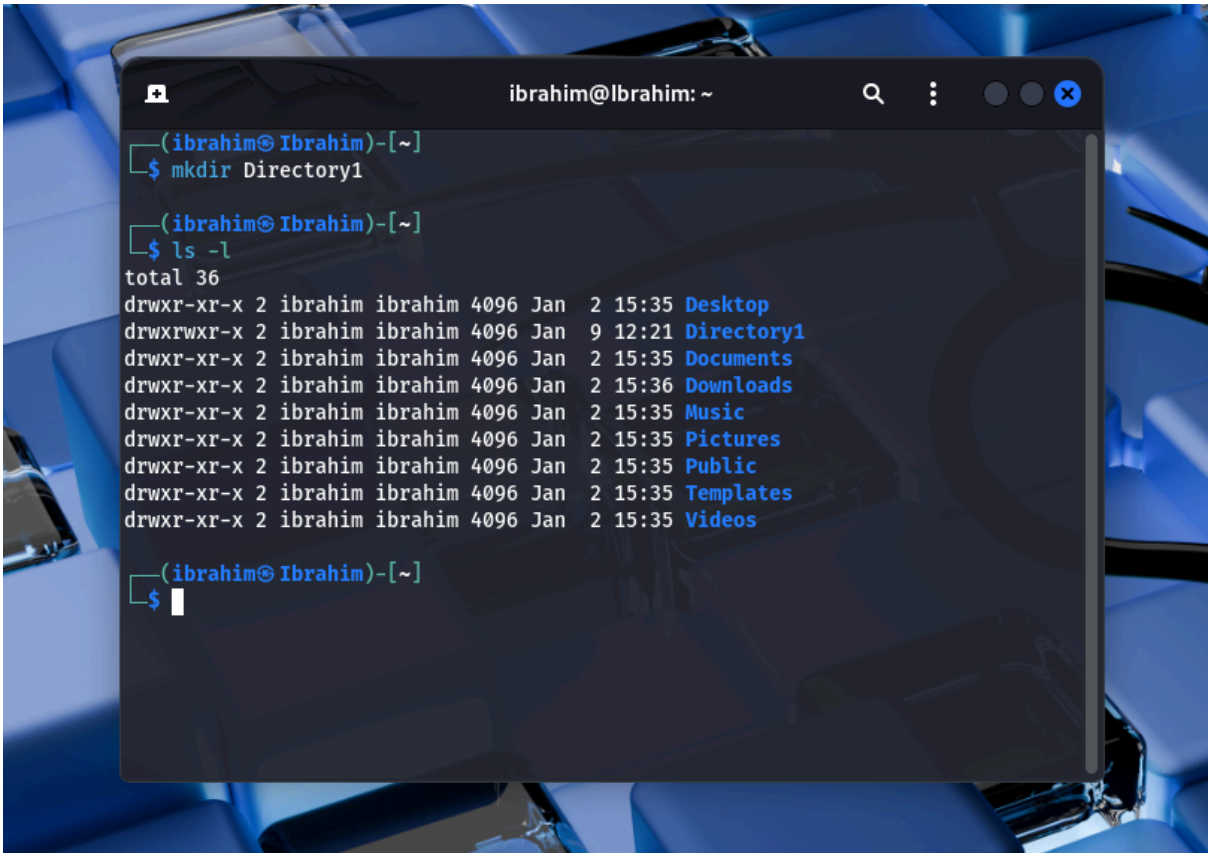
- **Dynamic Resolution:** Ensuring the interface scales correctly for clear, legible screenshots in this technical document.

Practical System Interaction (CLI Commands)

To document system proficiency, I performed the following operations via the Kali terminal to organize my assessment data.

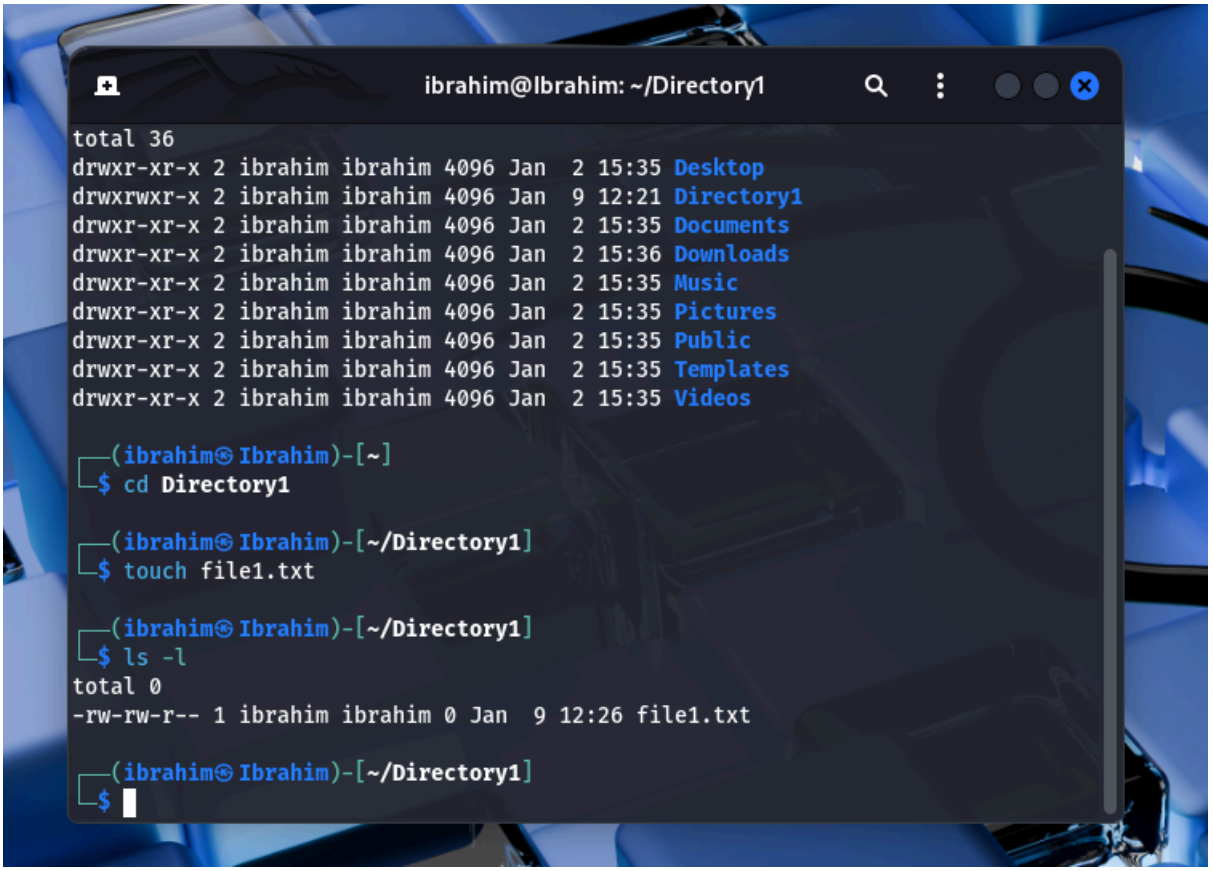
Creating Directory

- Command: ‘mkdir Directory name’
- Syntax: mkdir (Directory1)
- **Execution:**
    1. Open the terminal.
    2. Type mkdir Directory1 and press Enter.
    3. This command creates a directory named.
    4. Verification: ls -l will show the newly created directory.



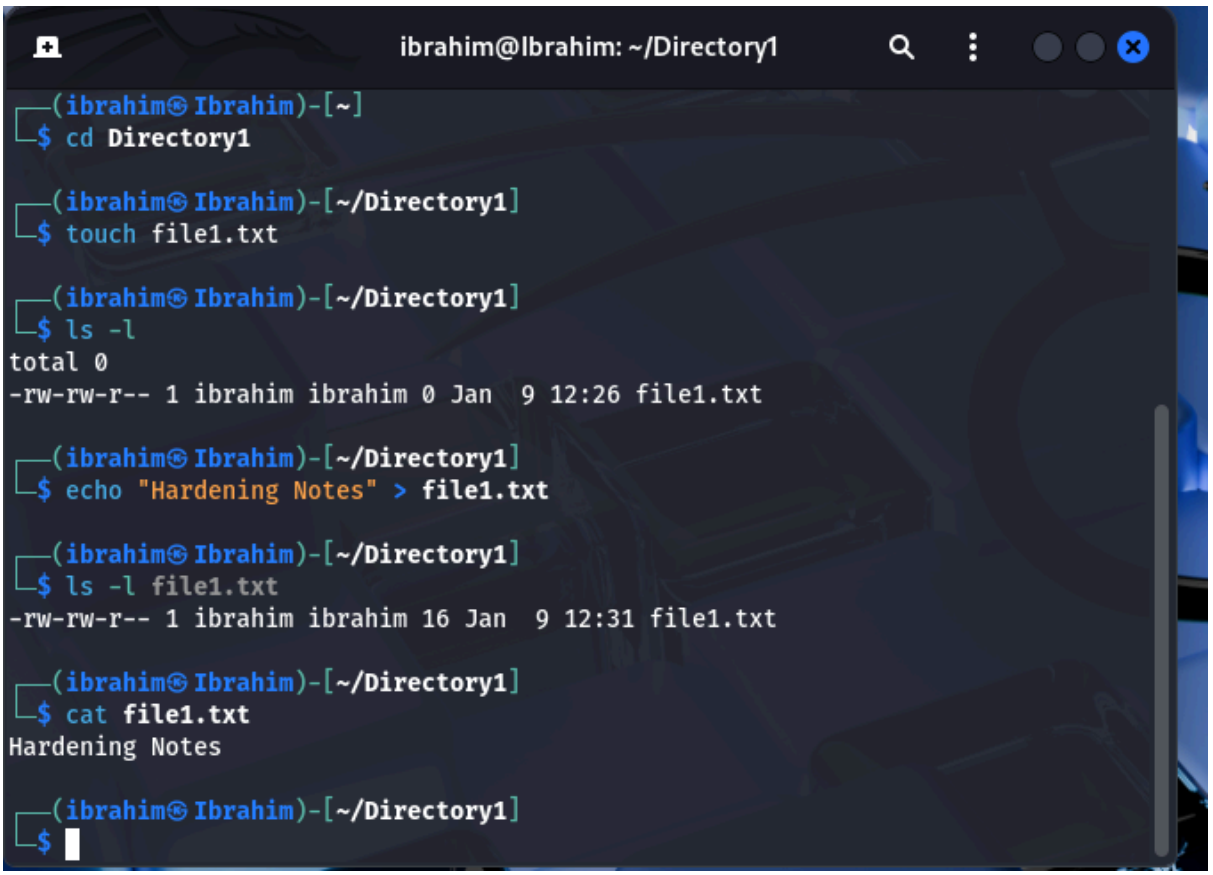
Creating File

- Command: touch file1.txt
- Syntax: touch [file1]
- Execution:
    1. In the terminal, type cd + name of dir and press Enter.
    2. This command changes to the specified directory (Directory1).
    3. Type touch + filename (file1)



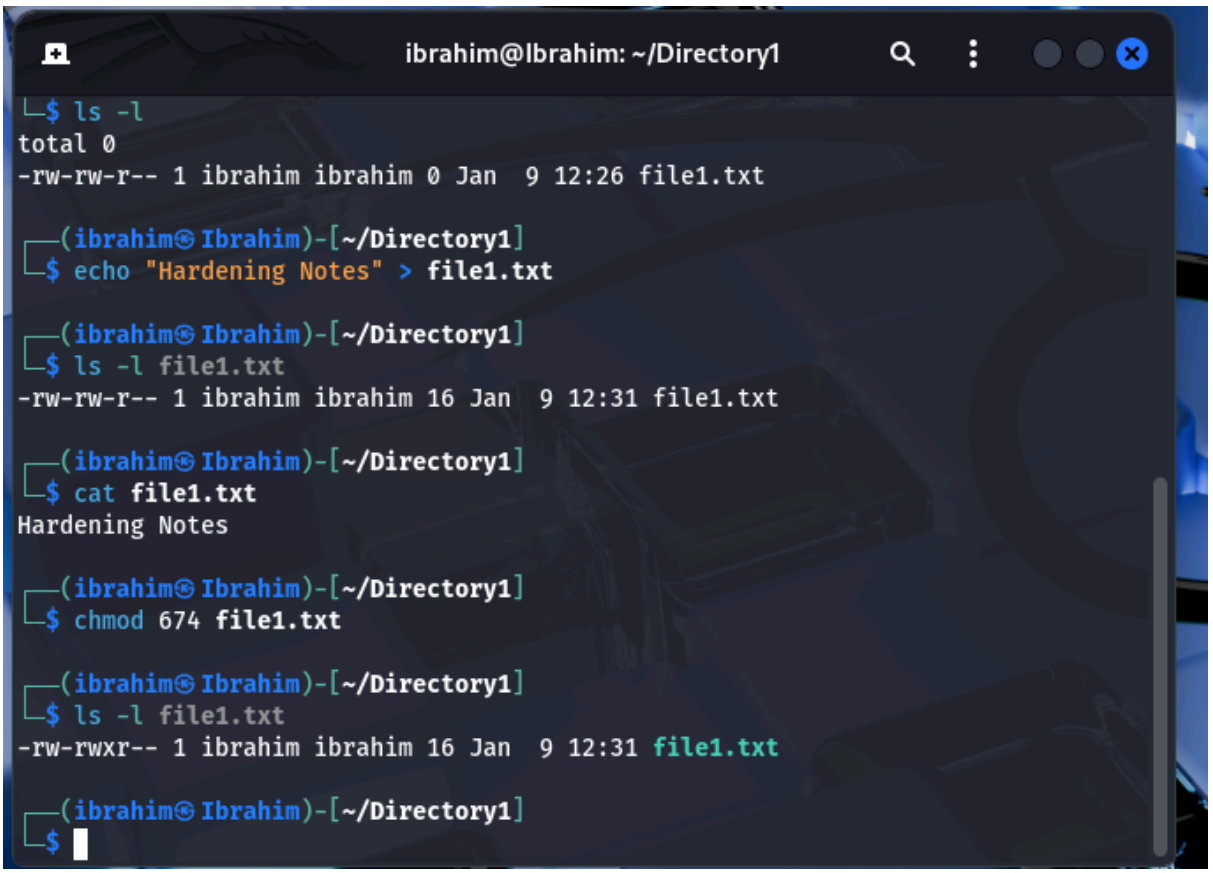
Content Population

- Command: echo >
- Syntax: echo “Hardening Notes” > file1.txt
- Execution:
    1. In the terminal, type echo "Hardening Notes" > file1.txt and press Enter.
    2. This command writes the specified string into the file created earlier.
    3. Type ls -l
    4. Type cat file1.txt to see content written



File permission modification

- Command: chmod 674 file1.txt
- Syntax: chmod [permissions] [file1]
- Execution:
    1. In the terminal, type chmod 674 file1.txt and press Enter.
    2. This command modifies the permissions of file1.txt using numeric mode.
    3. **Permission Breakdown:**
        - **6:** Owner (read and write: 4 + 2)
        - **7:** Group (read, write, and execute: 4 + 2 + 1)
        - **4:** Others (read-only: 4)



Question 3B: Website Hardening Recommendations

Following the comprehensive vulnerability assessment in Questions 1 and 2, I have developed the following technical hardening roadmap. These measures are designed to transform `testphp.vulnweb.com` from a legacy testing environment into a resilient, production-ready infrastructure.

Core Infrastructure Hardening

Recommendation	Linked Vulnerability	Implementation Objective
Enforce HTTPS/TLS 1.3	**Cleartext Transmission (2A, 2C)**	Encrypt all data-in-transit using a trusted TLS certificate. This prevents credential harvesting identified during Wireshark analysis.

Migrate to PHP 8.3	**EOL PHP 5.6.40 (2B)**	Decommission the unsupported PHP 5.6 branch to eliminate the **CVSS 10.0** attack surface. This ensures the server receives ongoing security patches.
Hardened Cookie Policy	**Insecure Cookies (2A)**	Modify the application backend to issue session tokens with `Secure`, `HttpOnly`, and `SameSite=Strict` flags.
Suppress Server Metadata	**Information Disclosure (1C, 2A)**	Configure the web server (Nginx/Apache) to suppress version strings (`server_tokens off;`).

Security Header Implementation Plan

To mitigate the browser-based attack vectors identified in the Burp Suite analysis, the following headers must be enforced at the server level:

- **HTTP Strict Transport Security (HSTS):**
  - **Command:** `Strict-Transport-Security: max-age=31536000; includeSubDomains`
  - **Purpose:** Forces the browser to connect via **HTTPS only**, neutralizing the Man-in-the-Middle (MitM) risk observed in Section 2C.
- **Content Security Policy (CSP):**
  - **Command:** `Content-Security-Policy: default-src 'self';`
  - **Purpose:** Prevents the execution of unauthorized scripts, directly addressing the risks associated with the **Insecure Cookies** found in Section 2A.
- **X-Frame-Options:**
  - **Command:** `X-Frame-Options: DENY`
  - **Purpose:** Protects users from clickjacking by preventing the site from being rendered inside an `<iframe>`.

In a professional security architecture, these measures follow the principle of **Defense in Depth**. By addressing the transport layer (HTTPS), the application layer (PHP Upgrade), and the client-side browser behavior (Security Headers), we create a multi-layered barrier.

As a result of these changes, even if a new vulnerability is discovered in the PHP code, the hardened headers and encrypted channel significantly increase the "cost of attack" for the adversary, likely forcing them to abandon the target in favor of less-hardened infrastructure.

Question 3C: Security Policy Design

Password Policy

Primary Principle: Integrity & Confidentiality

- **Complexity Requirements:** All user passwords must be a minimum of **14 characters** and include a combination of uppercase letters, lowercase letters, numbers, and special characters.
- **MFA Mandate:** Multi-Factor Authentication (MFA) must be enabled for all administrative interfaces and backend database access.
- **Prohibitions:** The use of "default" credentials or common dictionary words is strictly prohibited.
- **Rotation:** Passwords must be changed immediately if there is any suspicion of compromise, as identified in the traffic analysis phase.

Acceptable Use Policy (AUP)

Primary Principle: Confidentiality

- **Authorized Access:** Users shall only access the website and backend systems for legitimate business purposes using assigned credentials.
- **Data Protection:** Employees are prohibited from uploading sensitive company data to the web server unless it is stored in an encrypted directory with restricted permissions.
- **Software Integrity:** No unauthorized third-party scripts or legacy software may be installed without a formal security review.

Incident Reporting Policy

Primary Principle: Availability

- **Definition of Incident:** An incident includes any identified unauthorized access, service disruption (DoS), or evidence of "malicious" packets as observed in **Wireshark** captures.

- **Reporting Timeline:** All employees and contractors must report suspected security anomalies to the IT Security lead within **one hour** of discovery.
- **Evidence Preservation:** In the event of a breach, staff must not reboot or wipe systems until a "snapshot" or forensic image has been secured for analysis.

Policy Alignment Table

Policy	CIA Pillar	Business Justification
Password	**Integrity**	Ensures that only authorized personnel can modify website content.
AUP	**Confidentiality**	Prevents the accidental exposure of technical metadata to external scanners.
Incident	**Availability**	Minimizes downtime by ensuring rapid response to resource exhaustion or attacks.

Question 3D: Quantitative Risk Analysis

To provide a business-centric justification for the hardening measures proposed in Section 3B, I have performed a **Quantitative Risk Analysis**. This method uses mathematical formulas to estimate the financial impact of security risks, allowing management to make data-driven investment decisions.

Asset Selection: Customer Database

For this analysis, I have selected the **Customer Database** as the primary asset. This database contains user credentials, personal information, and session data. Its compromise represents the highest risk to the organization's reputation and legal standing.

Risk Calculations

To determine the total risk, we calculate three primary values:

1. **SLE (Single Loss Expectancy):**
    - **Formula:** AssetValue×ExposureFactor(EF)
    - **Calculation:** If the database asset is valued at **$10,000** (cost of data, recovery labor, and legal notification) and a single breach exposes 50% of its value:
    - **10,000×0.50=$5,000**
2. **ARO (Annual Rate of Occurrence):**
    - **Definition:** The estimated number of times this specific threat (e.g., a data breach) is expected to occur in one year.
    - **Assessment:** Given the **Critical (10.0)** vulnerabilities found in the PHP backend and the lack of encryption, a realistic ARO for this unhardened system is **2.0** (two successful breaches per year).
3. **ALE (Annual Loss Expectancy):**
    - **Formula:** SLE×ARO
    - **Calculation: $5,000×2.0=$10,000**

Influence on Security Investment Decisions

The calculated **ALE of $10,000** provides a clear financial ceiling for security spending. This value influences the organization's strategy in the following ways:

- **Cost-Benefit Analysis:** If the cost of implementing **HTTPS** and upgrading to **PHP 8.3** is $2,000, the organization saves $8,000 in potential annual losses. This makes the investment a high-priority "Green Light" for management.
- **Risk Mitigation vs. Acceptance:** If a security tool costs $15,000 per year (more than the ALE), it may be more financially sound to **accept** the risk or find a cheaper mitigation strategy, as the "cure" is more expensive than the "disease."
- **Resource Prioritization:** By calculating the ALE for multiple assets, the organization can direct its limited budget toward the highest-loss areas first, ensuring the most efficient use of security capital.